

Graph-Constrained Reasoning 技术报告

项目概述

项目名称: Graph-Constrained Reasoning (GCR)

论文: [Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models](#)

发表会议: ICML 2025

核心贡献

Graph-constrained Reasoning (GCR) 是一个创新性框架,旨在解决大语言模型(LLM)在知识图谱(KG)推理中的幻觉问题。该框架通过将知识图谱的结构化信息与大语言模型的非结构化推理能力相结合,实现了零幻觉的可信推理路径生成。

技术架构

1. 核心技术组件

1.1 KG-Trie 数据结构

项目实现了两种 Trie 数据结构用于高效的图索引:

- 标准 Trie** (`src/trie.py::Trie`): 基于字典的 Trie 实现
 - 支持动态添加序列
 - 提供高效的前缀查询
 - 支持 Trie 组合操作
- Marisa Trie** (`src/trie.py::MarisaTrie`): 基于 marisa-trie 库的内存优化版本
 - 使用字符编码优化内存占用
 - 支持最多 256,001 个 token
 - 缓存首层分支以提升性能
 - 适用于大规模知识图谱索引

核心实现:

```
# Trie 索引构建: 将图路径序列化为 token 序列
# 查询时根据前缀返回允许的下一个 token
def get(self, prefix_sequence: List[int]) -> List[int]:
    # 返回所有可能的下一个 token
```

1.2 Graph-Constrained Decoding

核心算法 (`src/graph_constrained_decoding.py`):

该模块实现了约束解码的核心逻辑,通过以下机制确保生成的推理路径严格遵循知识图谱结构:

- 约束标志检测**: 通过特殊的开始/结束 token 标识约束解码区域
- Token 过滤**: 在约束区域内,只允许生成 Trie 中存在的 token 序列
- 无缝集成**: 通过 `prefix_allowed_tokens_fn` 接口与 HuggingFace Transformers 集成

关键代码片段:

```
def allowed_tokens_fn(self, batch_id: int, sent: torch.Tensor):
    # 检查是否在约束解码区域
    constrained_flag, L_input = self.check_constrained_flag(sent)

    if constrained_flag:
        # 从 Trie 获取允许的 token
        allow_tokens = self.trie.get(sent.tolist()[L_input:])
        if len(allow_tokens) == 0:
            return self.all_tokens # 回退到无约束
    return allow_tokens
```

1.3 Graph Inductive Reasoning 图归纳推理 (The Judge)

这是 GCR 框架的 **Stage 2** 核心。由于 Stage 1 可能会生成多条合法的推理路径（Group Beam Search 的结果），该组件负责利用通用大模型的语义理解能力，对这些路径进行综合研判。

- **任务目标:** 解决“多路径冲突”和“路径语义理解”问题。Stage 1 保证了路径的**真实性**，Stage 2 负责保证答案的**准确性**。
- **流程:**
 1. **收集:** 将 Stage 1 生成的 Top-K 条路径（如 SwitchA -> Port1 -> Down 和 SwitchA -> Config -> Error）收集起来。
 2. **Prompt 构建:** 将这些结构化路径转化为自然语言 Prompt。
 3. **归纳:** 调用通用 LLM（如 GPT-4 或 Llama-3）分析哪条路径最符合用户问题的意图，并生成最终答案。

项目结构

```
graph-constrained-reasoning/
├─ src/                                # 核心源代码
│   ├── graph_constrained_decoding.py  # 约束解码核心算法
│   ├── trie.py                        # Trie 数据结构实现
│   ├── qa_prompt_builder.py           # 提示词构建
│   └─ llms/                           # LLM 集成层
│       ├── base_language_model.py
│       ├── base_hf_causal_model.py
│       ├── graph_constrained_decoding_model.py
│       └─ chatgpt.py
│   └─ model_adapter.py                # 多模型适配器
├─ utils/                              # 工具函数
│   ├── graph_utils.py                # 图操作工具
│   ├── qa_utils.py                   # QA 评估工具
│   └─ align_utils.py                 # 对齐工具
├─ workflow/                           # 完整工作流脚本
│   ├── build_graph_index.py          # 图索引构建
│   ├── build_shortest_path_index.py
│   ├── finetune_kg_specialized_llm.py # 模型微调
│   ├── predict_paths_and_answers.py   # Stage 1 推理
│   └─ predict_final_answer.py         # Stage 2 推理
└─ scripts/                            # Shell 脚本
```

```
|   |— build_graph_index.sh
|   |— train_kg_specialized_llm.sh
|   |— graph_constrained_decoding.sh
|   |— graph_inductive_reasoning.sh
|— accelerate_configs/          # 加速器配置
|— resources/                  # 资源文件(论文图片等)
|— pyproject.toml              # 项目配置
|— poetry.lock                 # 依赖锁定
|— README.md                   # 项目文档
```

3. 完整推理流程

Stage 1: Graph-Constrained Decoding

目标: 使用轻量级 KG-specialized LLM 生成 KG-grounded 推理路径

流程 (workflow/predict_paths_and_answers.py):

1. 加载问题数据集
2. 为每个问题构建 Trie 索引(基于 DFS 收集的路径)
3. 构建提示词 (使用 PathGenerationWithAnswerPromptBuilder)
4. 调用 generate_sentence() 进行约束解码
5. 使用 beam search 生成 top-k 推理路径和候选答案
6. 保存结果到 results/GenPaths/

关键特性:

- 零幻觉: 所有生成的路径都必然存在于知识图谱中
- 高效: 使用 Trie 数据结构实现 $O(\log n)$ 复杂度的 token 过滤
- 可扩展: 支持 group-beam search 生成多样化的路径

Stage 2: Graph Inductive Reasoning

目标: 使用通用 LLM 对多个推理路径进行归纳推理

流程 (workflow/predict_final_answer.py):

1. 加载 Stage 1 生成的推理路径和候选答案
2. 构建包含多个路径的提示词
3. 调用 GPT-3.5-turbo 或 GPT-4o-mini
4. 综合多个推理路径生成最终答案
5. 评估答案准确性

优势:

- 利用 LLM 的归纳推理能力
- 无需额外训练
- 可解释性强(提供完整推理路径)

前缀树

KG 中的路径有大量共享前缀

路径1: Barack Obama → birthplace → Honolulu → located_in → Hawaii
路径2: Barack Obama → birthplace → Honolulu → located_in → USA
路径3: Barack Obama → birthplace → Hawaii → type → US_State
路径4: Barack Obama → birthdate → 1961
路径5: Barack Obama → profession → Politician

用普通树或者列表存储 (每个节点存完整路径)

需要 5 个节点，每个存储完整路径
内存: $5 \times (\text{平均长度 } 4) = 20$ 个 token 存储空间
查询: 需要检查所有 5 个节点，时间复杂度: $O(N * M)$ ，其中 N 是路径数量，M 是路径平均长度

用 Trie 存储

```
graph TD
    root((root)) --> BO[Barack Obama]
    BO --> BP[birthplace]
    BO --> BD[birthdate]
    BO --> P[profession]
    BP --> H[Honolulu]
    BP --> HI[Hawaii]
    BD --> 1961[1961]
    P --> Pol[Politician]
    H --> LI[located_in]
    HI --> LI
    LI --> US[Hawaii]
    LI --> USA[USA]
    Pol --> USState[type]
    USState --> USState2[US_State]
```

所有路径共享这个节点!

共享的节点:

- "Barack Obama" 被5条路径共享 → 只存1次
- "birthplace" 被3条路径共享 → 只存1次
- "Honolulu" 被2条路径共享 → 只存1次

实际存储: 大约 12 个唯一节点

内存节省: 40%，时间复杂度: $O(M)$ - M 是前缀长度

Trie 的优势

```
# 场景: 已生成 [Barack, Obama, →, birthplace]
# 问题: 下一步能生成什么?

# 普通树/列表:
for path in all_paths: # 遍历 100,000 条路径
    if path[:4] == [Barack, Obama, →, birthplace]:
        candidates.add(path[4])
# 100,000 次比较

# Trie:
node = root['Barack']['Obama']['→']['birthplace']
candidates = node.children.keys() # 直接拿到子节点
# 4 次查找 + 1 次获取子节点
```

详细步骤分析

步骤1: 构建 KG 索引 (Trie)

代码位置

- `src/qa_prompt_builder.py` → `PathGenerationWithAnswerPromptBuilder.get_graph_index()`
- `workflow/build_graph_index.py`

1.1 子步骤: DFS 遍历 KG

输入:

```
{
  "q_entity": ["Barack Obama"], # 起始实体
  "graph": [ # KG 三元组列表
    ["Barack Obama", "place_of_birth", "Honolulu"],
    ["Barack Obama", "profession", "Politician"],
    ["Honolulu", "containedby", "Hawaii"],
    ...
  ]
}
```

处理:

```
# src/utils/graph_utils.py → dfs()
def dfs(graph, start_node_list, max_length=2):
    """
    从起始节点出发, 深度优先搜索所有长度 <= max_length 的路径
    """
    path_lists = set()
    for start_node in start_node_list:
        dfs_visit(start_node, [])
```

```

return list(path_lists)

# 示例执行：
# 起始: "Barack Obama"
# max_length = 2

# 找到的路径：
# 1跳: Barack Obama -> place_of_birth -> Honolulu
# 1跳: Barack Obama -> profession -> Politician
# 2跳: Barack Obama -> place_of_birth -> Honolulu -> containedby -> Hawaii
# ...

```

输出: 路径列表

```

paths_list = [
    [("Barack Obama", "place_of_birth", "Honolulu")],
    [("Barack Obama", "profession", "Politician")],
    [
        ("Barack Obama", "place_of_birth", "Honolulu"),
        ("Honolulu", "containedby", "Hawaii")
    ],
    ...
]
# 可能有 1万 ~ 10万+ 条路径

```

1.2 子步骤: 路径转字符串

输入: 路径三元组列表

处理:

```

# src/utils/utils.py → path_to_string()
def path_to_string(path: list) -> str:
    result = ""
    for i, (h, r, t) in enumerate(path):
        if i == 0:
            result += f"{h} -> {r} -> {t}"
        else:
            result += f" -> {r} -> {t}" # 后续跳省略头实体
    return result

# 示例：
path = [
    ("Barack Obama", "place_of_birth", "Honolulu"),
    ("Honolulu", "containedby", "Hawaii")
]
path_to_string(path)
# 输出: "Barack Obama -> place_of_birth -> Honolulu -> containedby -> Hawaii"

```

输出: 字符串列表

```
paths_list_str = [
    "Barack Obama -> place_of_birth -> Honolulu",
    "Barack Obama -> profession -> Politician",
    "Barack Obama -> place_of_birth -> Honolulu -> containedby -> Hawaii",
    ...
]
```

1.3 子步骤: Tokenize 路径

输入: 路径字符串列表

处理:

```
# 使用模型的 tokenizer 将字符串转为 token ID
tokenized_paths = tokenizer(paths_list_str,
                             padding=False,
                             add_special_tokens=False).input_ids

# 示例:
# "Barack Obama -> place_of_birth -> Honolulu"
# → [2045, 8084, 4287, 98623, 6254, 24671]
```

输出: Token 序列列表

```
tokenized_path_list = [
    [2045, 8084, 4287, 98623, 6254, 24671],
    [2045, 8084, 4287, 67890, 12121, 45678],
    [2045, 8084, 4287, 98623, 6254, 24671, 5555, 6666],
    ...
]

# 每条路径后加 EOS token
tokenized_path_list = [
    ids + [tokenizer.eos_token_id] for ids in tokenized_paths
]
```

1.4 子步骤: 构建 MarisaTrie

输入: Token 序列列表

处理:

```
# src/trie.py → MarisaTrie
trie = MarisaTrie(tokenized_path_list,
                  max_token_id=len(tokenizer) + 1)

# 内部结构 (概念图):
# {
#   2045: { # "Barack"
#     8084: { # "Obama"
#       4287: { # "->"
#         98623: { # "place_of_birth"
#           6254: { # "->"
```

```
#           24671: {EOS} # "Honolulu" + EOS
#           }
#           },
#           67890: {...} # "profession" 分支
#           }
#       }
#   }
# }
```

输出: MarisaTrie 对象

```
trie.get([])
# → [2045] # 第一个 token 只能是 2045 (Barack)

trie.get([2045, 8084, 4287])
# → [98623, 67890, ...] # "Barack Obama ->" 后可接的关系
```

步骤2: 构造 Prompt

代码位置

- src/qa_prompt_builder.py →
PathGenerationWithAnswerPromptBuilder.process_input()

输入

```
{
    "question": "Where was Barack Obama born?",
    "q_entity": ["Barack Obama"]
}
```

处理

```
# Zero-shot prompt template
ZERO_SHOT_PROMPT = """Reasoning path is a sequence of triples in the KG that
connects the topic entities in the question to answer entities. Given a question,
please generate some reasoning paths in the KG starting from the topic entities
to answer the question.

# Question:
{question}
# Topic entities:
{entities}
"""

# 填充
input_query = ZERO_SHOT_PROMPT.format(
    question="Where was Barack Obama born?",
    entities="Barack Obama"
)
```


输出

```
input_query = """Reasoning path is a sequence of triples in the KG that connects
the topic entities in the question to answer entities. Given a question, please
generate some reasoning paths in the KG starting from the topic entities to
answer the question.

# Question:
Where was Barack Obama born?
# Topic entities:
Barack Obama
"""
```

步骤3: LLM 图约束解码 (核心)

代码位置

- `src/graph_constrained_decoding.py` → `GraphConstrainedDecoding.allowed_tokens_fn()`
- `src/l1ms/graph_constrained_decoding_model.py` → `generate_sentence()`

3.1 子步骤: 准备模型输入

输入:

```
llm_input = """<|begin_of_text|><|start_header_id|>user<|end_header_id|>

Reasoning path is a sequence of triples in the KG...
# Question:
Where was Barack Obama born?
# Topic entities:
Barack Obama
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
"""

# (Chat template 格式化后)
```

处理:

```
# Tokenize
inputs = tokenizer(llm_input, return_tensors="pt")
input_ids = inputs.input_ids # Shape: [1, L]
# 例: [1, 128000, 271, 49, ..., 78191] (L=50 tokens)
```

3.2 子步骤: 创建约束函数

处理:

```
# src/graph_constrained_decoding.py
gcr = GraphConstrainedDecoding(
    tokenizer=tokenizer,
```

```

trie=trie, # 步骤1构建的 Trie
start_token_ids=None, # 不使用 <PATH> 标记
end_token_ids=None,
enable_constrained_by_default=False # 默认不约束
)

# 关键函数
def allowed_tokens_fn(batch_id: int, sent: torch.Tensor):
    """
    每生成一个 token 都会调用此函数

    Args:
        batch_id: beam 的索引
        sent: 已生成的完整 token 序列 [input_ids + generated_ids]

    Returns:
        allow_tokens: 允许生成的 token ID 列表
    """

    # 提取已生成的部分 (去掉 input)
    L_input = 50 # 输入长度
    generated = sent[L_input:] # 已生成的 token

    # 查询 Trie
    allow_tokens = trie.get(generated.tolist())

    if len(allow_tokens) == 0:
        # Trie 中没有匹配的路径 → 允许所有 token (退出约束)
        return list(range(len(tokenizer)))

    return allow_tokens

```

3.3 子步骤: Beam Search with Constraint

处理:

```

# HuggingFace generate 方法
res = model.generate(
    input_ids=input_ids, # [1, 50]
    generation_config=GenerationConfig(
        num_beams=10, # 10 个 beam
        num_return_sequences=10,
        num_beam_groups=10, # Group beam search
        diversity_penalty=1.0, # 鼓励多样性
        max_new_tokens=1024
    ),
    prefix_allowed_tokens_fn=gcr.allowed_tokens_fn, # 约束函数!
    return_dict_in_generate=True
)

```

逐步解码示例:

初始状态:

```
input_ids = [1, 128000, ..., 78191] (50 tokens, 结尾是 "assistant\n")
```

Beam 1:

Step 1: 生成第 1 个 token

- 当前: [input_ids]
- 调用 `allowed_tokens_fn(0, [input_ids])`
- `generated = []`
- `trie.get([]) → [2045]` (只能是 "Barack")
- 模型在允许列表 [2045] 中选择 → 生成 2045
- 现在: [input_ids, 2045]

Step 2: 生成第 2 个 token

- 当前: [input_ids, 2045]
- 调用 `allowed_tokens_fn(0, [input_ids, 2045])`
- `generated = [2045]`
- `trie.get([2045]) → [8084]` (只能是 "Obama")
- 生成 8084
- 现在: [input_ids, 2045, 8084]

Step 3: 生成第 3 个 token

- 当前: [input_ids, 2045, 8084]
- `generated = [2045, 8084]`
- `trie.get([2045, 8084]) → [4287]` (只能是 "->")
- 生成 4287
- 现在: [input_ids, 2045, 8084, 4287]

Step 4: 生成第 4 个 token

- 当前: [input_ids, 2045, 8084, 4287]
- `generated = [2045, 8084, 4287]`
- `trie.get([2045, 8084, 4287]) → [98623, 67890, 12345, ...]`
- 这里有多重选择! (place_of_birth, profession, ...)
- 模型根据概率选择 98623 (place_of_birth)
- 现在: [input_ids, 2045, 8084, 4287, 98623]

...继续直到生成完整路径和答案...

最终 Beam 1:

```
"Barack Obama -> place_of_birth -> Honolulu\n# Answer:\nHonolulu"
```

Group Beam Search 产生多样化结果:

Beam 1: Barack Obama -> place_of_birth -> Honolulu

Beam 2: Barack Obama -> place_of_birth -> Honolulu -> containedby -> Hawaii

Beam 3: Barack Obama -> place_of_birth -> Honolulu -> containedby -> Hawaii ->
type -> US_State

Beam 4: ... (其他路径)

...

Beam 10: ...

3.4 输出解码

输入: 生成的 token 序列

处理:

```
# res.sequences: [10, L_total] # 10 个 beam 的结果
# 去掉输入部分, 只保留生成的
generated_ids = res.sequences[:, input_ids.shape[1]:]

# Decode
responses = []
for seq in generated_ids:
    text = tokenizer.decode(seq, skip_special_tokens=True)
    responses.append(text)
```

输出:

```
responses = [
    "# Reasoning Path:\nBarack Obama -> place_of_birth -> Honolulu\n# Answer:\nHonolulu",
    "# Reasoning Path:\nBarack Obama -> place_of_birth -> Honolulu -> containedby -> Hawaii\n# Answer:\nHawaii",
    ...
]
```

步骤4: 解析输出

代码位置

- `workflow/predict_paths_and_answers.py` → `prediction()`

输入

```
prediction = [
    "# Reasoning Path:\nBarack Obama -> place_of_birth -> Honolulu\n# Answer:\nHonolulu",
    ...
]
```

处理

```
# 构建结果字典
result = {
    "id": "WebQSP_001",
    "question": "Where was Barack Obama born?",
    "prediction": prediction, # 保持原格式
    "ground_truth": ["Honolulu"],
    "ground_truth_paths": [
        "Barack Obama -> place_of_birth -> Honolulu"
    ],
    "input": llm_input
```

```

}

# 保存到 JSONL
with open('predictions.jsonl', 'a') as f:
    f.write(json.dumps(result) + '\n')

```

输出文件

```

{"id": "WebQSP_001", "question": "where was Barack Obama born?", "prediction":
["# Reasoning Path:\nBarack Obama -> place_of_birth -> Honolulu\n#
Answer:\nHonolulu", ...], "ground_truth": ["Honolulu"], ...}
{"id": "WebQSP_002", ...}
...

```

详细步骤说明

Step 1: 数据准备

代码位置: `main()` 函数开始部分

```

# 加载问答数据集
dataset = load_dataset(input_file, split=args.split)
# 示例: RoG-webqsp 测试集

# 加载Stage 1的路径
paths_datasets = []
with open(args.reasoning_path, "r") as f:
    for line in f:
        paths_datasets.append(json.loads(line))

# 合并
dataset = merge_path_result(dataset, paths_datasets)

```

输入示例:

```

# 原始数据集样本
{
    'id': 'WebQTest-1234',
    'question': 'Where was Barack Obama born?',
    'q_entity': ['Barack Obama'],
    'answer': ['Hawaii']
}

# Stage 1路径预测
{
    'id': 'WebQTest-1234',
    'prediction': [
        'Barack Obama -> born_in -> Hawaii',
        'Barack Obama -> born_in -> Hawaii',
        'Barack Obama -> spouse -> Michelle Obama',
        'Barack Obama -> born_in -> Hawaii'
    ]
}

```

```

}

# 合并后
{
    'id': 'webQTest-1234',
    'question': 'Where was Barack Obama born?',
    'predicted_paths': [
        'Barack Obama -> born_in -> Hawaii',
        'Barack Obama -> spouse -> Michelle Obama'
    ], # 注意: 去重后只有2条
    'answer': ['Hawaii']
}

```

Step 2: Prompt构建

代码位置: `PromptBuilder.process_input()`

Prompt模板:

```

SAQ_RULE_INSTRUCTION = """Based on the reasoning paths, please answer the given
question. Please keep the answer as simple as possible and only return
answers."""

GRAPH_CONTEXT = """Reasoning Paths:
{context}

"""

QUESTION = """Question:
{question}"""

```

构建过程:

```

def process_input(self, question_dict):
    question = question_dict["question"]

    # 1. 提取候选路径
    lists_of_paths = question_dict['predicted_paths']

    # 2. 构建问题部分
    input = self.QUESTION.format(question=question)
    # 结果: "Question:\nwhere was Barack Obama born?"

    # 3. 构建指令部分
    instruction = self.SAQ_RULE_INSTRUCTION

    # 4. 检查长度并截断路径
    context = self.check_prompt_length(
        other_prompt, lists_of_paths, self.maximun_token
    )

    # 5. 组装完整prompt
    input = self.GRAPH_CONTEXT.format(context=context) + input
    input = PROMPT_TEMPLATE.format(instruction=instruction, input=input)

```

```
return input
```

生成的Prompt示例:

Reasoning Paths:

Barack Obama -> born_in -> Hawaii

Barack Obama -> born_in -> Hawaii

Barack Obama -> spouse -> Michelle Obama

Question:

where was Barack Obama born?

Based on the reasoning paths, please answer the given question. Please keep the answer as simple as possible and only return answers.

Step 3: 长度检查与截断

代码位置: `PromptBuilder.check_prompt_length()`

```
def check_prompt_length(self, prompt, list_of_paths, maximun_token):
    all_paths = "\n".join(list_of_paths)
    all_tokens = prompt + all_paths

    # 如果长度合适, 直接返回
    if self.tokenize(all_tokens) < maximun_token:
        return all_paths
    else:
        # 超长: 随机打乱并逐条添加
        random.shuffle(list_of_paths)
        new_list_of_paths = []

        for p in list_of_paths:
            tmp_all_paths = "\n".join(new_list_of_paths + [p])
            tmp_all_tokens = prompt + tmp_all_paths
            if self.tokenize(tmp_all_tokens) > maximun_token:
                return "\n".join(new_list_of_paths)
            new_list_of_paths.append(p)
```

功能:

- 防止prompt超过模型的最大context长度
- 如果路径太多, 优先保留部分路径
- 随机打乱保证公平性

示例:

```
# 假设最大长度 = 4096 tokens
# 路径列表有100条, 总长度 = 5000 tokens

# 执行后:
# - 随机选择约80条路径
# - 总长度 < 4096 tokens
# - 既保证多样性, 又不超长
```

Step 4: LLM推理

代码位置: `make_prediction()` → `model.generate_sentence()`

```
# 将prompt输入到LLM
prediction = model.generate_sentence(input)
```

LLM内部处理 (以GPT-3.5为例) :

1. 理解问题: 识别出在问"出生地"
2. 分析路径:
 - 路径1: Barack Obama -> born_in -> Hawaii ✓ 相关
 - 路径2: Barack Obama -> born_in -> Hawaii ✓ 相关 (重复)
 - 路径3: Barack Obama -> spouse -> Michelle Obama ✗ 不相关
3. 归纳推理:
 - 多条路径指向 "Hawaii"
 - "born_in" 关系直接回答了问题
 - "spouse" 关系与问题无关
4. 生成答案: Hawaii

输出示例:

```
# 简单答案
prediction = "Hawaii"

# 或带解释的答案
prediction = "Hawaii\n\nBased on the reasoning paths, Barack Obama was born in Hawaii."
```

Step 5: 答案提取与保存

代码位置: `make_prediction()` 返回部分

```
result = {
    "id": "WebQTest-1234",
    "question": "Where was Barack Obama born?",
    "prediction": "Hawaii",
    "ground_truth": ["Hawaii"],
    "input": "Reasoning Paths:\n..."
}

# 写入文件
fout.write(json.dumps(result) + "\n")
```

输出文件格式 (predictions.jsonl) :


```
{ "id": "q_001", "question": "...", "prediction": "Hawaii", "ground_truth": ["Hawaii"], "input": "..."}
{ "id": "q_002", "question": "...", "prediction": "USA", "ground_truth": ["United States"], "input": "..."}
{ "id": "q_003", "question": "...", "prediction": "1961", "ground_truth": ["1961"], "input": "..."}

```

RoG-webqsp 测试结果

1 评估指标

指标	值	含义
Accuracy	52.94%	正确预测的样本数占总样本数的比例，衡量模型整体预测的准确性
Hit	79.55%	至少有一个正确答案被预测到的样本比例，衡量模型找到正确答案的能力
F1 Score	56.19%	精确率和召回率的调和平均值，综合衡量模型的精确性和全面性
Precision	74.52%	预测为正例的样本中实际为正例的比例，衡量模型预测结果的可靠性
Recall	52.94%	实际为正例的样本中被正确预测为正例的比例，衡量模型对正确答案的覆盖程度

2 预测示例

示例 1

- **问题:** what does jamaican people speak?
- **推理路径:**

```
Jamaica -> location.country.languages_spoken -> Jamaican Creole English Language

```

- **预测答案:**

```
Jamaican Creole English Language
English

```

- **正确答案:**

```
Jamaican English, Jamaican Creole English Language

```

示例 2

- **问题:** what did james k polk do before he was president?
- **推理路径:**

James K. Polk -> people.person.profession -> Politician
James K. Polk -> government.politician.government_positions_held -> m.04j5sk8
-> government.government_position_held.office_position_or_title -> Governor
of Tennessee

- 预测答案:

Politician
Governor of Tennessee

- 正确答案:

United States Representative, Governor of Tennessee, Speaker of the United
States House of Representatives

示例 3

- 问题: who plays ken barlow in coronation street?
- 推理路径:

Coronation Street -> tv.tv_program.program_creator -> Tony Warren ->
tv.tv_program_creator.programs_created -> Coronation Street
Coronation Street -> tv.tv_program.program_creator -> Tony Warren ->
people.person.profession -> Screenwriter

- 预测答案:

William Roache

- 正确答案:

William Roache

示例 4

- 问题: where is jamarcus russell from?
- 推理路径:

JaMarcus Russell -> people.person.nationality -> United States of America
JaMarcus Russell -> people.person.place_of_birth -> Mobile

- 预测答案:

United States of America
Mobile

- 正确答案:

Mobile

示例 5

- **问题:** where was george washington carver from?
- **推理路径:**

```
George Washington Carver -> people.person.place_of_birth -> Diamond ->  
location.location.containedby -> Missouri  
George Washington Carver -> people.person.place_of_birth -> Diamond ->  
location.location.containedby -> United States of America
```

- **预测答案:**

```
Missouri  
United States of America
```

- **正确答案:**

```
Diamond
```

3 结果分析

整体性能: 模型在RoG-webqsp测试集上取得了52.94%的准确率和79.55%的命中率，表明模型能够较为准确地从知识图谱中推理出答案，同时保持了较高的召回率（52.94%）和精确率（74.52%）。